

Triggers

07_etapa2_triggers.sql — três regras que o banco aplica sozinho

Trigger e função de trigger

No PostgreSQL um trigger não carrega código. Ele aponta para uma função declarada com RETURNS TRIGGER, e é essa função que recebe as variáveis OLD, NEW e TG_OP. Por isso cada item deste documento tem duas partes: a função fn_ e o trigger trg_ que a dispara. Uma consequência prática é que a mesma função pode servir a mais de um trigger, o que o terceiro caso aproveita.

trg_check_sobreposicao_escala

BEFORE INSERT OR UPDATE em ESCALA. Impede que o mesmo residente apareça no mesmo dia e turno em duas unidades diferentes.

O que a UNIQUE da Etapa 1 deixa passar

A restrição do esquema original é UNIQUE(id_unidade, dia_semana, turno, id_residente). Para ela, o residente 11 na segunda pela manhã na Enfermaria A e o mesmo residente 11 na segunda pela manhã na UTI são duas linhas diferentes, porque a unidade faz parte da chave. As duas passam. O trigger fecha essa brecha, e o efeito combinado é que um residente tem no máximo um plantão por dia e turno.

Seção 6 do 10_etapa2_verificacao.sql

```
-- Recusado pelo trigger, aceito pela UNIQUE
INSERT INTO Escala (dia_semana, turno, id_preceptor, id_residente, id_unidade)
VALUES ('segunda', 'manha', 7, 11, 3);

ERROR:  Residente 11 já está escalado em segunda manha na unidade "Enfermaria A".
HINT:   Um residente não pode cobrir duas unidades no mesmo turno.
```

A regra vale só para o residente

O preceptor continua podendo supervisionar vários residentes no mesmo dia e turno, em unidades diferentes. O enunciado permite isso de forma explícita, e o seed da Etapa 1 tem o caso: o preceptor 6 aparece na segunda pela manhã supervisionando o residente 11 na Enfermaria A e o residente 12 na UTI. Um trigger que olhasse o preceptor recusaria dados válidos.

Por que BEFORE e não AFTER

A verificação precisa acontecer antes da gravação, para que a linha nunca chegue à tabela. Um trigger AFTER também conseguiria recusar, levantando exceção e provocando rollback, mas faria o banco escrever para depois desfazer. BEFORE é a forma natural de validar.

A consulta interna exclui a própria linha com id_escala diferente de NEW.id_escala. Isso importa no UPDATE, para que a escala não conflite consigo mesma. No INSERT o valor de NEW.id_escala já existe, porque o PostgreSQL avalia o padrão da coluna IDENTITY antes de disparar os triggers BEFORE.

trg_audita_atendimento

AFTER INSERT OR UPDATE OR DELETE em ATENDIMENTO. Grava em auditoria_atendimento o estado anterior e o posterior de cada linha alterada.

Trecho de fn_audita_atendimento

```
INSERT INTO Auditoria_Atendimento
(id_atendimento, operacao, usuario, dados_antigos, dados_novos)
VALUES (NEW.id_atendimento, 'UPDATE', session_user, to_jsonb(OLD), to_jsonb(NEW));
```

A linha inteira em JSON

to_jsonb(OLD) e to_jsonb(NEW) convertem a linha completa em um único valor JSONB. A alternativa seria uma coluna de auditoria para cada coluna de atendimento, e nesse caso acrescentar um campo à tabela exigiria mexer na auditoria também. Com JSONB, a coluna id_unidade que a Etapa 2 criou passou a ser auditada sem nenhuma alteração no trigger.

A interface aproveita esse formato: a aba Auditoria compara os dois JSON campo a campo e mostra o que mudou, sem saber de antemão quais colunas a tabela tem.

session_user e não current_user

A coluna usuario recebe session_user, o usuário que abriu a conexão. current_user muda com SET ROLE e dentro de rotina SECURITY DEFINER, então apontaria para outro papel nas situações em que alguém usa esses recursos. Numa auditoria, isso esconderia quem agiu.

Por que AFTER

O trigger registra o que o banco aceitou. Em BEFORE, uma linha recusada por constraint depois da execução do trigger deixaria registro de uma alteração que nunca aconteceu.

A linha de DELETE sobrevive ao atendimento

auditoria_atendimento.id_atendimento não tem chave estrangeira. Com ON DELETE CASCADE, apagar o atendimento levaria embora o registro da remoção. Sem cascata, a FK impediria a remoção. Nenhuma das duas serve, e a coluna ficou sem referência declarada.

O seed da Etapa 2 já deixa a tabela com quinze linhas: dez UPDATE, do preenchimento da unidade nos atendimentos antigos, e cinco INSERT, dos atendimentos novos. Nenhuma delas foi escrita pela aplicação.

trg_atualiza_media_procedimentos

AFTER INSERT em PROCEDIMENTO_REALIZADO. Recalcula procedimento.media_tempo_procedimento com a média de tempo_real_minutos daquele procedimento em todos os atendimentos.

O que o enunciado pede e o que isso deixa de fora

O enunciado especifica AFTER INSERT. Só com isso, a média fica desatualizada quando alguém corrige o tempo de um registro ou apaga a linha, e a API da Etapa 1 permite as duas coisas: PUT e DELETE em /procedimentos-realizados. Um valor derivado que só reage a inserções deixa de ser confiável na primeira correção.

A solução foi manter o trigger exatamente como pedido e criar um segundo sobre a mesma função:

```
CREATE TRIGGER trg_atualiza_media_procedimentos
  AFTER INSERT ON Procedimento_Realizado
  FOR EACH ROW EXECUTE FUNCTION fn_atualiza_media_procedimentos();

CREATE TRIGGER trg_atualiza_media_procedimentos_ud
  AFTER UPDATE OR DELETE ON Procedimento_Realizado
  FOR EACH ROW EXECUTE FUNCTION fn_atualiza_media_procedimentos();
```

O trigger com o nome do enunciado existe e faz o que foi especificado. O segundo é acréscimo nosso, com nome diferente para não haver dúvida sobre o que foi pedido e o que foi decisão de projeto.

UPDATE que troca o procedimento

Se um registro passa do procedimento 2 para o procedimento 5, duas médias mudam: a do procedimento que perdeu a linha e a do que ganhou. A função compara OLD.id_procedimento com NEW.id_procedimento e recalcula os dois quando são diferentes. O recálculo em si fica numa função separada, fn_recalcula_media_procedimento, chamada com PERFORM.

Custo

Cada procedimento realizado provoca um AVG sobre todas as realizações daquele procedimento. Com o volume de um trabalho de disciplina isso é irrelevante. Num sistema real com milhões de linhas, a forma usual seria manter soma e contagem em colunas próprias e atualizá-las de forma incremental, ou abandonar a coluna derivada e calcular na leitura.

Verificação

As seções 4, 5 e 6 do 10_etapa2_verificacao.sql exercitam os três triggers. As partes que gravam ficam entre BEGIN e ROLLBACK, então o roteiro pode ser executado quantas vezes for necessário sem sujar o banco. Um exemplo do que ele verifica:

Seção 4

```
BEGIN;
  UPDATE Procedimento_Realizado SET tempo_real_minutos = 60
  WHERE id_procedimento = (SELECT id_procedimento FROM Procedimento
                           WHERE codigo = 102)
    AND id_atendimento = (SELECT id_atendimento FROM Atendimento
                           WHERE data_hora = TIMESTAMP '2026-06-15 08:30');
  -- media de Coleta de sangue: 10.60 antes, 20.20 depois
  SELECT nome, media_tempo_procedimento FROM Procedimento WHERE codigo = 102;
ROLLBACK;
```

Na interface, a aba Auditoria mostra o histórico e explica como gerar linhas novas. A aba Procedimentos mostra a coluna de média, que muda ao registrar um procedimento realizado na aba correspondente.